* Github copilot chat (extension).

21st October - 2026 - wednesday :-

## * Decision making :-

* Boolean Values :- A Boolean is a data type with exactly 2 values (True / False).

-> They are case-sensitive : it should be written exactly as shown below.

```
# Boolean values
is_active = True
is_disabled = False
```

* comparison operators : compare two values & return a Boolean result.

== equals to , != not equals to , > greater than , < less than , >= greater than or equal to.

* String comparison :- To check if two strings are equal.

-> we can ~~even~~ even compare alphabetically to see which comes first.

-> They are case-sensitive by default so capital letter != small letters.

```
product_a = "10 faux flowers"
product_b = "1 Faux Flowers"

print(product_a == product_b)     # False

print(product_a > product_b)      # True
```

-> single '=' assigns a value

-> double '==' compares values

* if, else, elif :- conditional flow statements :- (conditional logic)

-> if runs when a condition is true.
-> else runs when a condition is false.
-> elif runs when extra condition's are true.

↓ if first match wins ; then others are skipped.

# Always order your conditions from most specific to least specific. If not then the least specific will get true & most specific condition may get skipped.

* not, and, or (logical operators) :-

Eg :- if not office_closed:
    print("office is open")

# combining multiple operators

if not office_closed and not office_weekend:
    print("office is officially opened")

# use parenthesis if you want a specific sequence of running.
# Python runs code inside parenthesis 1st & first outside later.

@ 24th - october - 2026 - Saturday :-

# when you need to repeat an action you don't need to write the statement again & again.
A loop repeats a block of code automatically, making your program shorter & easier to maintain.
They repeat as long as a condition is true.
(for, while, break, continue).
# for loop, which repeats a set number of times.
# while loop, which repeats as long as a condition is true.
# control loop behavior with break & continue.

* Loops: loops automate repetative tasks by running the same block of code again. Python gives you 2 ~~options~~ choices a for loop - when you know the repetition & a while loop when repetetion depends on condition.

-> How range works (start, stop, step): range can take a start value, stop value & a step. Eg: range (1, 10, 2)

(1) → (3) → (5) → (7) → (9)

Syntax: for i in range(5): # i = 0, 1, 2, 3, 4
    print(i)

Output: 0
1
2
3
4

* How while loop works:- A while checks the condition before every repetition. If the condition stays true everytime, python doesn't stop & freeze the program by infinite loop.

Eg:-
```
count = 0
while count < 5:
    print(count)        # count = 0, 1, 2, 3, 4
    count += 1
```

Output:
```
0
1
2
3
4
```

Eg:-
```
health = 3
while health > 0:
    print(f"Player is alive. Health: {health}")
    health -= 1         # modify the condition so the loop will
                        eventually stop.
```

* Choose the loop that fits:

| for loop | while loop |
|---|---|
| - Knows repetition count<br>- Automatic exit.<br>- counter tracks repetitions<br>- visits each collection item | Variable state controls repetition<br>stops after an event.<br>Condition controls exit<br>repetition count is unknown. |

* Break statement: immediately ends a loop.

Eg:-
```
for count in range(10):
    if count == 5:
        break                    # exit loop; skip print
    print(count)
print("Loop ended.")
```

# Break exits the loop early. (It skips the next code).

* Continue: It skips the remaining code in a current iteration, then starts the next one.

Eg:-
```
for count in range(6):
    if count == 3:               # skip 3; continue loop.
        continue
    print(count)
```

# Continue bypasses the print statement but loop keeps running.
# Continue skips one iteration.
# use continue when one iteration should be ignored without ending the whole loop.

* Break in while loop:-

```
while True:
    user_input = input("Type 'exit' to stop the loop:")
    if user_input.lower() == "exit":
        break
    print(f"you entered : {user_input}")

print("Goodbye")
```

# Break ends a while true loop.

* Continue in a while loop needs extra care:-

```
current_slot = 0
while current_slot < 5:
    current_slot += 1  # crucial: increments happen
                         right at the start of the loop

    if current_slot == 3:
        print(f"skipping item slot {current_slot}")
        continue  # skips the rest of this loop & jumps
                    back to the top

    # This processing code is skipped when current_slot is 3
    print(f"Successfully processed item in slot {current_slot}")

print("Inventory scan complete")

    # update the condition before continue.
```

* Loops can have else block:-

```
color = "blue"
for i in range(3):
    user_input = input("Guess color: ")
    if user_input.lower() == color:
        print("You guessed it!")
        break                    # break reached: skip else
else:                            # no break: run else
    print(f"Sorry, the correct answer is {color}")


    # else runs only without break
```